

Generating Sudoku with 60 language models

One identical prompt, sent to 60 models across 8 vendors. Every generated game was rendered, validated, played to a win in a real browser, and published here. This is a relative, point-in-time comparison on a single task — read the methodology and limitations before quoting a number.

Disclosure

This study is produced by **PlayPrompt**, a beta platform that generates browser games from natural-language prompts using third-party models. We state that up front. PlayPrompt has **no model of its own** in this comparison and **no fixed default** — we run studies like this to decide which models to offer, so our incentive is an accurate result, not promoting any one model.

To keep the comparison verifiable, every model received the same prompt under identical conditions and **every generated game is published here** to play and re-score. Our fairness practices follow the literature on benchmark conflicts of interest (see References).

The study was run and graded with the help of **Claude Code**, an Anthropic agent. Logic and playability verdicts are decided by execution — a solver and each game's own win signal — but the visual pass was made by a Claude-based reviewer and was not blinded. That is a second conflict of interest (an Anthropic model helping grade games, including games written by Anthropic models), so every screenshot is published for independent re-grading; see Threats to Validity.

Abstract

We measured whether current language models can turn one natural-language prompt into a complete, correct, playable browser game. Sudoku was the probe because correctness is machine-verifiable. Sixty models from eight vendors received the same prompt; each output was scored on a frozen rubric: did it **deliver**, encode a **valid puzzle that can be driven to a recognized win**, and **visually render a usable board**. Of 60 models, **41** delivered a working game and **23** produced a board that was both logically correct and cleanly rendered; 1 produced a valid but reduced (4×4) variant, and the remainder were flawed, broken, or not Sudoku. The strongest result is

methodological: logic-only checking produces false positives — at least one model computed a correct solution while rendering no grid at all — so both execution-based and pixel-level verification are required. We publish every generated game, every screenshot, the end-user prompt, and the per-model data. Findings are relative and conditional on this prompt, these model snapshots, and this date.

Motivation

PlayPrompt generates games with AI, so we have a direct, practical interest in where current models succeed and fail at turning a prompt into working software. This study is one probe in that effort, and we chose Sudoku deliberately:

- **Well known, and almost certainly in training data.** That makes it a test of *generation and rendering* rather than novel reasoning. We treat the training-data overlap as a scope limit, not an advantage (see Threats to Validity).
- **Simple in the right way.** A static logic puzzle has no real-time action, physics, or moving parts, so it isolates correctness, layout, and a single clear win condition — not animation or game-feel.
- **Automatically verifiable.** A Sudoku is either a valid grid with a solvable puzzle and a detectable win, or it is not — which lets us grade by execution rather than by opinion.

It is the first in a planned series; later probes will add what Sudoku deliberately leaves out — real-time action, physics, and multi-step state.

The prompt

Every model received the same end-user instruction, byte-for-byte — not shared with any vendor in advance, and deliberately short to mirror real usage:

```
create a sudoku game with jazz music.
```

That sentence is wrapped in a fixed system instruction — identical for all 60 models — that asks for a single self-contained HTML file (inline CSS/JS, no external libraries, mobile-friendly) plus a small configuration block, and an automatic step expands the sentence into the model's user message. We describe rather than reproduce that instruction to avoid publishing proprietary prompt text; it was identical across all models, is available to model vendors on request for verification, and its possible effect on results is noted under Threats to Validity. The jazz soundtrack is added by a separate post-step and is not part of what the model writes.

Methodology

Models

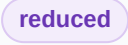
Sixty current text models across eight vendors — Anthropic, OpenAI, Google, xAI, DeepSeek, Mistral, MiniMax, and Alibaba (Qwen) — were enumerated in advance; the full roster with exact model IDs and per-model outcomes is in Appendix A. The model list, the prompt, the rubric, and the "best-by" definitions were fixed before the run.

Generation

Each model was driven through PlayPrompt's generation path via a per-request model override, so all 60 faced identical conditions — same prompt, same enhancement step, same system instruction, same packaging. The only variable was the text model that wrote the game. Models were run with default sampling settings; one attempt per model. Generation time and token usage were recorded per model.

Scoring rubric (pre-stated)

Each delivered game was assigned exactly one verdict against definitions fixed before grading:

Verdict	Definition
 clean	Renders a real, non-garbled 9×9 grid with visible 3×3 box divisions and printed givens;
 reduced	Valid and playable and driven to a win, but a reduced/non-standard variant (e.g. 4×4) r
 flawed	Renders, but the puzzle is invalid, the input is broken, or there is nothing to play.
 broken	No usable board renders, or the game is not Sudoku.

Solution uniqueness was not part of the clean bar (a proper Sudoku has exactly one solution; our bar was a valid, winnable board). Where uniqueness was measured and failed, the game is kept as clean but flagged  — see Threats to Validity.

How games were evaluated — tools and routines

The whole pipeline — issuing the generation requests, running the test harness, reviewing the screenshots, aggregating the data, and drafting this report — was orchestrated with **Claude Code**, Anthropic's agentic coding tool. Games were tested in a real headless browser: **Chromium driven by Playwright** (Node), at a fixed 1000×1150 viewport, after waiting for fonts to load and nudging the viewport to trigger any canvas sizing.

Stage 1 — execution / logic. For each game the harness loads the page, starts it (clicking a difficulty or Start control, or the canvas), and reads the puzzle: for DOM-rendered boards it extracts the digit in every cell by walking the rendered elements and clustering them into the N×N grid; for canvas-rendered boards it reads the puzzle from the game's in-page JavaScript state. A backtracking solver checks that the givens form a valid Sudoku (no repeated digit in any row, column, or 3×3 box) and that a solution

exists, then drives that full solution into the board through the game's own controls — selecting each empty cell and entering its digit — and records whether the game's own code signals a win. A game that cannot be solved, or never registers the win, fails this stage.

Stage 2 — pixels / visual. The harness screenshots each started game; the images were reviewed (by a vision-capable agent and by the author) against four criteria: a real grid is present, the 3×3 boxes are visibly divided, the given numbers are printed, and the layout is not garbled. Stage 2 exists because Stage 1 is not sufficient: one model (`grok-4.3`) passed every execution check — valid solver, correct win logic — yet rendered no grid at all. Only the screenshot caught it. Generated software has to be observed, not just trusted.

Cost and features

Estimated text-generation cost = recorded token usage × each vendor's published list price (asset costs excluded, to isolate the text model). Prices were read from each vendor's public pricing page on 2026-06-23; a small number (Alibaba/Qwen) are estimates and are flagged as such. Feature counts are the number of distinct interactive elements catalogued per game and are approximate.

► Glossary

Results

Attrition funnel

Models considered	60
Delivered a working game	41
Correct and cleanly-rendered 9×9	23
Reduced (valid 4×4) variant	1
Flawed / broken / not-Sudoku	17

Every rate in this study is reported with its denominator. The 19 not-delivered models (60–41) failed at generation for mixed reasons — rejected output, rate-limits, or timeouts — itemised per model in Appendix A; infrastructure failures (rate-limit/timeout) are labelled as such and not counted against the model's capability.

The 23 clean games — playable, sorted by cost

Each row links to the actual game (it runs in your browser) and its screenshot.  marks a puzzle that is valid and winnable but not uniquely solvable; estimated prices are noted in the methodology.

Model	Gen time	Tokens (in/out)	E
gemini-2.5-flash-lite	36s	10243/6808	\$
deepseek-v4-flash	122s	15305/16672	\$
qwen3-vl-plus	128s	13255/9919	\$
qwen3-coder-flash	68s	10012/6787	\$
devstral-medium-latest	130s	10180/6742	\$
qwen3.5-plus	198s	10571/13175	\$
deepseek-v4-pro	366s	11181/17558	\$
MiniMax-M2.7-highspeed	165s	15327/13471	\$
magistral-small-latest	49s	13319/9807	\$
qwen3.6-plus	245s	12816/14911	\$
gemini-3-flash-preview	35s	7716/7240	\$
gemini-2.5-flash	47s	8900/10281	\$
gpt-5.4-mini	46s	10026/6711	\$
claude-haiku-4-5-20251001	46s	11737/8014	\$
qwen3-coder-next	37s	10973/7804	\$
gemini-3.5-flash	76s	11791/15151	\$
gemini-3.1-pro-preview	99s	9036/12353	\$
claude-sonnet-4-6	131s	13923/10204	\$
gpt-5.4	89s	14263/11004	\$
claude-opus-4-5-20251101	108s	13788/10100	\$
claude-opus-4-6	151s	15364/11626	\$
claude-opus-4-8	102s	18257/12868	\$
claude-opus-4-7	148s	19231/13786	\$

Best by dimension

Reported as separate dimensions, not a single composite ranking. Magnitudes are observed values on this run.

FASTEST gemini-3-flash-preview 35s · clean 9×9	CHEAPEST gemini-2.5-flash-lite \$0.0037 est. · clean 9×9	MOST FEATURES qwen3.6-plus 20 catalogued
CLEAN-GAME COST RANGE \$0.0037 – \$0.4408 ≈118× spread		

Cost did not predict quality: the single most expensive run produced an *invalid* puzzle (see below), while the cheapest clean game cost a fraction of a cent. Generation time ranged 35–366s among clean games (median 102s).

Reduced / non-standard variant

Model	What it built	Verdict	Screenshot	Play
qwen3.7-max	A 4×4 "lounge" Sudoku (digits 1–4, 2×2 boxes) — valid, playable, driven to a win, but a reduced grid rather than standard 9×9.	reduced		▶ play

Where models failed

All delivered-but-not-clean games are listed here, by a single rule applied to every vendor — nothing was quietly dropped. Two failure modes are worth foregrounding: a game can render perfectly yet encode an **invalid puzzle** (e.g. `gpt-5.2-pro`, `qwen3.5-flash`), or pass every execution check yet **render nothing usable** (`grok-4.3`). Neither is caught without both QA stages.

Model	Verdict	What went wrong	Scr
qwen3.5-flash	flawed	Renders a clean 9×9, but the puzzle is invalid — duplicate digits among the printed givens within columns; contradictory as printed.	
qwen3.7-plus	flawed	A 4×4 variant that loads fully pre-filled with the completed solution — there is nothing to play.	

Model	Verdict	What went wrong	Scr
qwen3.6-flash	broken	The board never paints — a runtime error aborts drawing on every frame, leaving the play area empty.	—
claude-sonnet-4-5-20250929	flawed	Easy and Medium puzzles are valid and winnable, but the Hard difficulty generates an invalid board.	—
gpt-5.2-pro	flawed	Most generated boards are invalid Sudoku — duplicate digits within rows/columns (no valid solution). This was also the most expensive run.	—
gpt-5.3-codex	flawed	Valid, well-structured puzzle, but a gameplay/input issue blocked a clean win in testing.	—
MiniMax-M2.7	flawed	Valid, solvable puzzle, but the number-input mechanism is broken.	—
codestral-latest	flawed	The puzzle generator produces mathematically invalid solutions (a digit is reused within a unit).	—
mistral-large-2512	flawed	Number pad and decorative elements render on top of the board; the input box overlaps cells.	—
mistral-large-latest	flawed	Valid puzzle, but the input mechanism is broken — numbers cannot be entered reliably.	—
mistral-small-latest	flawed	The grid and 3×3 lines render very faintly and a floating control is mispositioned.	—
gemini-2.5-pro	broken	Produced a different game entirely (a falling-fruit arcade game), not Sudoku.	—
gemini-3.1-flash-lite-preview	broken	No win-detection logic — the puzzle cannot be completed.	—
MiniMax-M2	broken	The win-check contains a logic error; the game is unwinnable.	—

Model	Verdict	What went wrong	Scr
MiniMax-Text-01	broken	Renders a 9×9 grid with input, but it is not Sudoku — there are no given clues.	—
devstral-latest	broken	Renders and accepts input, but the puzzle generator is fundamentally broken (invalid puzzles).	—
grok-4.3	broken	No usable grid renders — only a stray line fragment and a colored blob, no given numbers. The internal solver and win-logic are correct, which is exactly why a logic-only check would wrongly pass it; the pixels reveal the failure.	—

Results by vendor

Clean 9×9 games as a share of games delivered. This is a descriptive roll-up of this run, not a general verdict on any vendor; each cell carries its denominator.

Vendor	Models tested	Delivered	Clean 9×9	Clean rate
DeepSeek	2	2	2	100%
Anthropic	8	7	6	86%
Google	9	7	5	71%
Alibaba (Qwen)	10	9	5	56%
OpenAI	10	4	2	50%
Mistral	10	7	2	29%
MiniMax	7	4	1	25%
xAI	4	1	0	0%

Discussion

Read these as relative, conditional observations — "under this one prompt, this rubric, on this date" — not absolute capability claims.

- **Three gates are all load-bearing.** Of 41 delivered games, only 23 were both correct and cleanly rendered. Failures split across invalid puzzles, broken input, and broken rendering, so evaluating generated software means driving it and inspecting it, not reading or trusting it.
 - **Cost did not predict quality.** The most expensive run produced an invalid puzzle; several of the cheapest games were clean. Buyers optimising for this kind of task should not assume the premium tier wins.
 - **Coding-tuned variants over-performed for their price.** The fast, cheap clean games came disproportionately from vendors' coder-oriented models, consistent with this being, underneath the theme, a code-generation task.
 - **Specification interpretation varies.** Given one ambiguous sentence, some models built a reduced 4×4 variant rather than standard 9×9 — valid, but not what most users expect. Single-sentence prompts leave real degrees of freedom.
 - **The top tier was reliable end-to-end; the long tail was not.** Several smaller or older models delivered something that looked right but failed on play or on render — the gap a one-glance demo would miss.
-

Threats to validity

Construct

One Sudoku prompt is not a measure of general game-generation ability. The task is "produce a self-contained HTML game that satisfies a fixed formatting and configuration contract," so a verdict blends Sudoku reasoning, HTML/JS coding, and contract-adherence. The automatic enhancement step and the configuration contract are identical for every model but may suit some better than others — a model marked failed may have mishandled the contract rather than the puzzle.

Internal

One attempt per model; model outputs are non-deterministic, so a re-run may differ and small gaps may be run-to-run noise. The per-request enhanced brief varied per call and was not individually retained at run time (a logging gap we are closing). Some Stage-1 failures were rate-limits or timeouts — infrastructure, not capability — and are labelled as such. Stage-2 grading involves human/visual judgment.

External

Sudoku only; these specific model snapshots; this date. Hosted model endpoints drift over time, so exact numbers are not guaranteed to reproduce.

Conclusion

Small N, single run, no statistical significance testing or confidence intervals. Feature counts are approximate; some costs are estimates. We avoid significance claims.

Grader independence

Logic and playability verdicts are execution-based — a deterministic solver and each game's own win signal decide them, not a model's opinion. The Stage-2 visual judgments, however, were made by a Claude-based agent (and the author) and were **not blinded** to which model produced each screenshot; since some contestants are Anthropic models, this is a conflict of interest. The visual check is a coarse "does a usable grid render" pass that is hard to skew, and all screenshots are published so the visual grades can be independently — and blindly — re-checked.

Contamination

Sudoku is heavily represented in training data. This study measures generation and rendering of a familiar artifact under one framing, not novel reasoning, and small prompt changes can flip outcomes.

Future work

Run each model several times to separate a real difference from run-to-run noise; blind the visual grading; and extend beyond Sudoku to games that need real-time action, physics, and multi-step state — the dimensions this probe deliberately omits.

Reproducibility & artifacts

Everything needed to re-run or re-score this study is published here:

- The verbatim end-user prompt (above). The fixed system instruction is described in The prompt and is available to model vendors on request.
- Every generated game, playable: /games/ (30 self-contained HTML files).
- Every screenshot: /assets/.
- The per-model results as data: results.csv · results.json.
- This entire report as a single file: PDF download.
- The full model roster and per-model outcomes (Appendix A).

To reproduce: enumerate each vendor's current text models; send every model the same end-user sentence under identical generation settings; for each output, run a headless-browser check that renders the board, validates the puzzle, drives the solution to a win, and screenshots the result; price by recorded tokens × the vendor's published rate.

Corrections welcome. If you believe a model was mis-run, send a better prompt or a correction and we will re-run and update, recording the change here. This is v1.0 (run date 2026-06-23); material changes will be versioned and dated.

Appendix A — full roster & per-model outcome

All 60 models, by vendor, with the Stage-1 outcome and (for delivered games) the final verdict. Infrastructure failures are labelled.

Anthropic (8 models)

Model ID	Stage-1 outcome	Verdict
claude-fable-5	did not deliver	—
claude-haiku-4-5-20251001	delivered	clean
claude-opus-4-5-20251101	delivered	clean
claude-opus-4-6	delivered	clean
claude-opus-4-7	delivered	clean
claude-opus-4-8	delivered	clean
claude-sonnet-4-5-20250929	delivered	flawed
claude-sonnet-4-6	delivered	clean

OpenAI (10 models)

Model ID	Stage-1 outcome	Verdict
gpt-5.2	did not deliver	—
gpt-5.2-codex	did not deliver	—
gpt-5.2-pro	delivered	flawed
gpt-5.3-codex	delivered	flawed
gpt-5.4	delivered	clean
gpt-5.4-mini	delivered	clean
gpt-5.4-nano	rejected (invalid output)	—
gpt-5.4-pro	did not deliver	—
gpt-5.5	did not deliver	—
gpt-5.5-pro	did not deliver	—

Google (9 models)

Model ID	Stage-1 outcome	Verdict
<code>gemini-2.0-flash</code>	did not deliver	—
<code>gemini-2.5-flash</code>	delivered	clean
<code>gemini-2.5-flash-lite</code>	delivered	clean
<code>gemini-2.5-pro</code>	delivered	not_sudoku
<code>gemini-3-flash-preview</code>	delivered	clean
<code>gemini-3-pro-preview</code>	did not deliver	—
<code>gemini-3.1-flash-lite-preview</code>	delivered	broken
<code>gemini-3.1-pro-preview</code>	delivered	clean
<code>gemini-3.5-flash</code>	delivered	clean

xAI (4 models)

Model ID	Stage-1 outcome	Verdict
<code>grok-4.20-0309-non-reasoning</code>	rejected (invalid output)	—
<code>grok-4.20-0309-reasoning</code>	rejected (invalid output)	—
<code>grok-4.20-multi-agent-0309</code>	did not deliver	—
<code>grok-4.3</code>	delivered	broken

DeepSeek (2 models)

Model ID	Stage-1 outcome	Verdict
<code>deepseek-v4-flash</code>	delivered	clean
<code>deepseek-v4-pro</code>	delivered	clean

Mistral (10 models)

Model ID	Stage-1 outcome	Verdict
<code>codestral-latest</code>	delivered	flawed

Model ID	Stage-1 outcome	Verdict
devstral-latest	delivered	broken
devstral-medium-latest	delivered	clean
magistral-medium-latest	did not deliver	—
magistral-small-latest	delivered	clean
ministral-14b-latest	rejected (invalid output)	—
ministral-8b-latest	rejected (invalid output)	—
mistral-large-2512	delivered	flawed
mistral-large-latest	delivered	flawed
mistral-small-latest	delivered	flawed

MiniMax (7 models)

Model ID	Stage-1 outcome	Verdict
MiniMax-M2	delivered	broken
MiniMax-M2.1	rejected (invalid output)	—
MiniMax-M2.5	rejected (invalid output)	—
MiniMax-M2.7	delivered	flawed
MiniMax-M2.7-highspeed	delivered	clean
MiniMax-M3	did not deliver	—
MiniMax-Text-01	delivered	not_sudoku

Alibaba (Qwen) (10 models)

Model ID	Stage-1 outcome	Verdict
qwen3-coder-flash	delivered	clean
qwen3-coder-next	delivered	clean
qwen3-coder-plus	rejected (invalid output)	—
qwen3-vl-plus	delivered	clean

Model ID	Stage-1 outcome	Verdict
qwen3.5-flash	delivered	flawed
qwen3.5-plus	delivered	clean
qwen3.6-flash	delivered	broken
qwen3.6-plus	delivered	clean
qwen3.7-max	delivered	reduced
qwen3.7-plus	delivered	flawed

References

1. Singh et al. (2025). *The Leaderboard Illusion*. arXiv:2504.20879 — arxiv.org/abs/2504.20879
2. DeepSource. *Every AI code-review vendor benchmarks itself, and wins*. deepsources.com/blog/ai-code-review-benchmarks
3. Willison, S. (2025). *Pelican on a bicycle — one-shot SVG across models*. simonwillison.net
4. Artificial Analysis. *Intelligence Index methodology*. artificialanalysis.ai
5. Aider. *Polyglot coding leaderboard*. aider.chat/docs/leaderboards
6. Epoch AI. *Benchmarking hub*. epoch.ai/benchmarks/about
7. Si et al. (2024). *Design2Code*. arXiv:2403.03163 — arxiv.org/abs/2403.03163

Cost figures use each vendor's published list prices, accessed 2026-06-23. Model and product names are used to identify the systems tested.

About PlayPrompt

This study was produced by **PlayPrompt**, an agentic AI platform that turns a single prompt into a complete, ready-to-play browser game — generating the gameplay, the music, and, when needed, the visual assets. In this study only the underlying text model was varied; the enhancement, packaging, and soundtrack were identical across all 60 models. The 30 playable links above are real platform output.

Disclaimer

Independent, point-in-time study for informational purposes; single prompt, single run per model, on the date shown. Not a comprehensive evaluation of any model or vendor. Claude, GPT, Gemini, Grok, DeepSeek, Qwen, MiniMax, Mistral and other names are trademarks of their respective owners, used nominatively to identify the systems tested; PlayPrompt is independent and is not affiliated with, sponsored by, or endorsed by any of them. Costs are estimates from published list prices and may not reflect current pricing. Results may not reproduce exactly due to model non-determinism. Findings describe specific generated artifacts in this test, not a general judgment of any company. Provided "as is", without warranty.

Independent generation study · v1.0 · run date 2026-06-23 · 41 games generated, 30 published as playable · produced by PlayPrompt.